Name of Course	**Bachelor of Computer Application (BCA)**
Semester	**V**
Name of Subject	**Web Development and PHP Programming**
Subject Code	**BCA-502**
Marks	**75 Marks**
Lectures	**50 Lectures**

Unit I

1. Introduction to PHP

1.1 Introduction to PHP , History and Features of PHP
1.2 Installation & Configuration of PHP
1.3 Embedding PHP code in Your Web Pages
1.4 Understanding PHP, HTML, and White Space. Writing Comments in PHP
1.5 Sending Data to the Web Browser
1.6 Data types in PHP, Keywords in PHP
1.7 Using Variables, Constants in PHP
1.8 Expressions in PHP
1.9 Operators in PHP

1.1 Introduction to PHP , History and Features of PHP:

Introduction to PHP:

PHP (Hypertext Preprocessor) is a widely-used **open-source server-side scripting language** especially suited for web development. It is embedded within HTML and is used to create dynamic web pages and applications.

History of PHP:

1994: Created by **Rasmus Lerdorf** to track visits to his online resume.
1995: Released as **Personal Home Page Tools (PHP Tools)**.
1997: PHP 3 was released, officially named **PHP: Hypertext Preprocessor**.
2000 onward: Major versions (PHP 4, 5, 7, and 8) were introduced with improved features and performance.

Features of PHP:

1. **Open Source –** Free to use and has a large community.
2. **Cross-Platform –** Runs on various operating systems like Windows, Linux, macOS.
3. **Database Support –** Works well with MySQL, PostgreSQL, Oracle, etc.
4. **Embedded in HTML –** PHP code can be easily inserted within HTML.
5. **Server-side Execution –** Processes data on the server and sends output to the browser.

1.2 Installation & Configuration of PHP:

To run PHP scripts, we need a **web server**, **PHP**, and optionally a **database** like MySQL. These can be installed separately or together using packages like **XAMPP**, **WAMP**, or **MAMP**.

1. Using XAMPP (Windows/Linux/Mac):

Download XAMPP from the official website (apachefriends.org).
Run the installer and follow the setup instructions.
XAMPP installs **Apache (web server)**, **PHP**, and **MySQL** together.
After installation, open the **XAMPP Control Panel** and start **Apache** and **MySQL**.

2. Manual Installation (Windows Example):

Download PHP from the official website (php.net).
Install a **web server** (like Apache or Nginx).
Configure the server to work with PHP by editing config files (e.g., `httpd.conf` for Apache).
Set environment variables (like adding PHP to system PATH).

3. Configuration of PHP:

The main configuration file is **`php.ini`**.
You can modify settings like:

`upload_max_filesize` – for file uploads
`memory_limit` – maximum memory a script can use
`error_reporting` – level of error messages shown

After changes, **restart the server** to apply them.

4. Testing Installation:

Create a file called **`test.php`** with the code:

php

```
<?php phpinfo(); ?>
```

Save it in the web directory (e.g., `htdocs` in XAMPP).
Open a browser and go to `http://localhost/test.php`.

1.3 Embedding PHP code in Your Web Pages:

PHP can be easily embedded within HTML to create **dynamic web pages**. This means PHP code is written inside an HTML file using special tags.

1. PHP Tags:

PHP code is placed between the `<?php` and `?>` tags.

Example:

php

```
<?php
    echo "Hello, World!";
?>
```

2. Basic HTML with Embedded PHP:

html

```
<!DOCTYPE html>
<html>
<head>
```

```
        <title>PHP Example</title>
    </head>
    <body>
        <h1>Welcome!</h1>
        <p>
            <?php
                echo "Today is " . date("l");
            ?>
        </p>
    </body>
    </html>
```

In this example, PHP generates the current day dynamically.

3. File Extension:

Save the file with a **`.php`** extension (e.g., `index.php`).
PHP code will only be executed on a server with PHP installed.

4. Output:

The browser only sees the **HTML output** of the PHP code.
Any logic, calculation, or database interaction is done **server-side**.

5. Importance:

Embedding PHP allows for **dynamic content**, like showing usernames, dates, or data from a database.

1.4 Understanding PHP, HTML, and White Space. Writing Comments in PHP:

1. PHP and HTML Together:

PHP is usually embedded inside HTML to create dynamic web pages.
HTML handles structure and layout, while PHP handles logic and data.

Example:

```
    php
        <html>
        <body>
            <h1>
                <?php echo "Welcome to my website!"; ?>
            </h1>
        </body>
        </html>
```

2. White Space in PHP:

White space (spaces, tabs, newlines) in PHP is generally **ignored** by the parser.
It is used to **improve readability** and format code clearly.

Example:

```
    php
```

```
<?php
    $x = 5;
    $y = 10;
    echo $x + $y;
?>
```

The spaces and newlines do not affect how the code runs.

Writing Comments in PHP:

3 Types of Comments:

1. Single-line comment (using //):

php

```
// This is a single-line comment
```

2. Single-line comment (using #):

php

```
# Another way to write a single-line comment
```

3. Multi-line comment (using /* ... */):

php

```
/*
    This is a multi-line comment
    spanning multiple lines
*/
```

Comments are used to explain code, making it easier to understand and maintain.

1.5 Sending Data to the Web Browser:

In PHP, **sending data to the web browser** means displaying output (like text, HTML, or variables) that the user can see on the webpage.

1. Using `echo` Statement:

`echo` is the most common way to send output.

php

```
<?php
    echo "Hello, World!";
?>
```

This sends **"Hello, World!"** to the browser.

2. Using `print` Statement:

`print` works similarly to `echo`, but returns a value (1).

php

```
<?php
    print "Welcome to PHP!";
?>
```

3. Sending HTML with PHP:

You can send HTML tags using `echo`:

php

```
<?php
    echo "<h1>This is a heading</h1>";
?>
```

This will display a heading in the browser.

4. Sending Variables:

PHP can send variable data to the browser:

php

```
<?php
    $name = "Ali";
    echo "Hello, " . $name;
?>
```

5. Importance:

Sending data is essential for displaying results, messages, or dynamic content like user names, product lists, etc.

1.6 Data types in PHP, Keywords in PHP:

1. Data Types in PHP:

PHP supports different data types to store various kinds of values:

a. Scalar Data Types:

Integer – Whole numbers (e.g., `10`, `-5`)
Float (Double) – Decimal numbers (e.g., `3.14`, `-2.5`)
String – Sequence of characters (e.g., `"Hello"`)
Boolean – True or false (`true`, `false`)

b. Compound Data Types:

Array – Collection of values (e.g., `$arr = [1, 2, 3];`)
Object – Instance of a class

c. Special Data Types:

NULL – Represents no value
Resource – Reference to external resources like database connections

2. Keywords in PHP:

Keywords are **reserved words** that have special meaning in PHP.
They **cannot be used as variable names**, function names, or identifiers.

Examples of PHP Keywords:

`if`, `else`, `while`, `for`, `function`, `return`, `echo`, `class`, `public`, `private`, `var`, `try`, `catch`, `new`, `switch`, `case`, `default`, `break`, `continue`, `true`, `false`, `null`

Example:

php

```
<?php
    $name = "Ali"; // string
    $age = 25;     // integer
    echo "Name: " . $name;
?>
```

1.7 Using Variables, Constants in PHP:

1. Variables in PHP:

Variables are used to **store data** that can change during script execution.
In PHP, variables start with a **`$` symbol**.

Rules for variables:

Must start with a letter or underscore (`_`)
Cannot start with a number
Are **case-sensitive**

Example:

php

```
<?php
    $name = "Ali";
    $age = 20;
    echo "Name: " . $name;
?>
```

2. Constants in PHP:

Constants are **fixed values** that **do not change** during script execution.
Use the **`define()`** function to create a constant.
Constants **do not have a `$` symbol**.

Example:

php

```
<?php
    define("SITE_NAME", "MyWebsite");
    echo SITE_NAME;
?>
```

Key Differences:

Feature	Variable	Constant
Symbol	Starts with `$`	No symbol
Changeable	Yes	No
Defined by	Assignment(`=`)	`define()` function

1.8 Expressions in PHP:

An **expression** in PHP is any combination of **variables, values, operators, and functions** that produces a result or value.

1. Types of Expressions:

a. Arithmetic Expressions:

Used to perform mathematical operations.

php

```
<?php
    $x = 10;
    $y = 5;
    $result = $x + $y; // 15
?>
```

b. String Expressions:

Used to combine strings.

php

```
<?php
    $greeting = "Hello " . "World!";
?>
```

c. Logical Expressions:

Used to make decisions (returns true or false).

php

```
<?php
    $x = 10;
    $y = 5;
    $check = ($x > $y) && ($y > 0); // true
?>
```

d. Assignment Expressions:

Assigns values using `=`.

php

```
<?php
    $a = 20;
?>
```

2. Importance of Expressions:

Expressions are the **building blocks** of PHP code.

They are used in **calculations, conditions, loops, and function calls**.

3. Example:

php

```
<?php
    $price = 100;
```

```
$tax = 0.1;
$total = $price + ($price * $tax);
echo "Total price: " . $total;
?>
```

1.9 Operators in PHP:

Operators are symbols used to perform operations on variables and values. PHP supports several types of operators for tasks like arithmetic, comparison, logic, assignment, etc.

1. Types of Operators:

a. Arithmetic Operators:

Used for basic math operations.

Operator	Description	Example
`+`	Addition	`$a + $b`
`-`	Subtraction	`$a - $b `
`*`	Multiplication	`$a * $b `
`/`	Division	`$a / $b `
`%`	Modulus (Remainder)	`$a % $b `

b. Assignment Operators:

Used to assign values to variables.

Operator	Example	Meaning
`=`	`$a = 5`	Assigns 5 to $a
`+=`	`$a += 2`	$a = $a + 2
`-=`	`$a -= 3`	$a = $a - 3

c. Comparison Operators:

Used to compare values (returns true or false).

Operator	Description	Example
`==`	Equal to	`$a == $b`
`!=`	Not equal to	`$a != $b `
`>`	Greater than	`$a > $b `
`<`	Less than	`$a < $b `
`>=`	Greater or equal	`$a >= $b `

d. Logical Operators:

Used in conditional statements.

Operator	Description	Example
`&&`	AND	`($a > 5 &&  $b < 10)`
`||`	OR	`($a > 5 || $b < 10) `
`!`	NOT	`!($a == $b) `

2. Example:

php

```
<?php
    $x = 10;
    $y = 5;
    if ($x > $y && $x != 0) {
        echo "Valid numbers";
    }
?>
```

Unit II

2. Programming with PHP

2.1 Conditional statements: if, if-else, switch, The ? Operator
2.2 Looping statements: while Loop, do...while Loop, for Loop
2.3 Arrays in PHP: Introduction- What is Array?
2.4 Types of Arrays: Indexed Vs. Associative arrays, Multidimensional arrays
2.5 Creating Array, Accessing Array, Manipulating Arrays, Displaying arrays…..use of for.. each as loop
2.6 Using Array Functions
2.7 Including and Requiring Files- use of Include() and Require()
2.8 Implicit and Explicit Casting in PHP

2.1 Conditional statements: if, if-else, switch, The ? Operator:

Conditional Statements in PHP:

Conditional statements allow PHP to make **decisions** and execute certain blocks of code **based on conditions**.

1. `if` Statement:

Executes a block of code **if the condition is true**.

php

```
<?php
    $age = 18;
    if ($age >= 18) {
        echo "You can vote.";
    }
?>
```

2. `if-else` Statement:

Provides an **alternative block** if the condition is false.

php

```
<?php
    if ($age >= 18) {
        echo "Adult";
    } else {
        echo "Minor";
    }
?>
```

3. `switch` Statement:

Used to **select one block of code from many** options.

php

```
<?php
    $day = "Monday";
    switch ($day) {
        case "Monday":
            echo "Start of the week";
```

```
                break;
            case "Friday":
                echo "Weekend is near";
                break;
            default:
                echo "Normal day";
        }
    ?>
```

4. Ternary (`?`) Operator:

A **short-hand** way to write an `if-else` condition.

Syntax:

php

```
(condition) ? value_if_true : value_if_false;
```

Example:

php

```
<?php
    $marks = 75;
    $result = ($marks >= 40) ? "Pass" : "Fail";
    echo $result;
?>
```

2.2 Looping statements: while Loop, do...while Loop, for Loop:

Looping Statements in PHP:

Loops are used to **execute a block of code repeatedly** as long as a specific condition is true.

1. `while` Loop:

Repeats code **as long as the condition is true**.

php

```
<?php
    $i = 1;
    while ($i <= 5) {
        echo $i . " ";
        $i++;
    }
?>
```

Checks condition before executing the loop.

2. `do...while` Loop:

Executes the code **at least once**, then checks the condition.

php

```
<?php
    $i = 1;
```

```
        do {
            echo $i . " ";
            $i++;
        } while ($i <= 5);
    ?>
```

Always runs the loop once, even if condition is false.

3. `for` Loop:

Best when the number of repetitions is **known**.

php

```
    <?php
        for ($i = 1; $i <= 5; $i++) {
            echo $i . " ";
        }
    ?>
```

All loop steps (initialization, condition, increment) are in one line.

Summary Table:

Loop	Condition Check	Best Use
`while`	Before loop	Unknown repetitions
`do…while`	After first run	At least one execution needed
`for`	Before loop	Known number of repetitions

2.3 Arrays in PHP: Introduction- What is Array?

What is an Array?

An **array** in PHP is a special variable that can hold **multiple values** under a **single name**. Arrays are useful when you want to store and manage a list of items (like names, numbers, etc.).

1. Why Use Arrays?

To store a collection of data.
Makes code more organized and easier to manage.
Allows looping through values.

2. Types of Arrays in PHP:

a. Indexed Arrays (Numeric Index):

Uses numbers (starting from 0) as keys.

php

```
    <?php
        $fruits = array("Apple", "Banana", "Mango");
        echo $fruits[1]; // Outputs: Banana
    ?>
```

b. Associative Arrays:

Uses **named keys** instead of numbers.

php

```
<?php
    $person = array("name" => "Ali", "age" => 25);
    echo $person["name"]; // Outputs: Ali
?>
```

c. Multidimensional Arrays:

Arrays within arrays (like a table or matrix).

php

```
<?php
    $marks = array(
        array(85, 90),
        array(78, 88)
    );
    echo $marks[0][1]; // Outputs: 90
?>
```

3. Advantages of Arrays:

Store large amounts of data.
Easy to sort, loop, and search.
Supports both numbers and string keys.

2.4 Types of Arrays: Indexed Vs. Associative arrays, Multidimensional arrays:

Types of Arrays in PHP:

PHP supports **three main types of arrays**:

1. Indexed Arrays:

Uses **numeric indexes** (starting from 0).
Stores values in a linear order.

Example:

php

```
<?php
    $colors = array("Red", "Green", "Blue");
    echo $colors[1]; // Output: Green
?>
```

Best when values are ordered and don't need labels.

2. Associative Arrays:

Uses **named keys** instead of numbers.
Stores data in **key-value pairs**.

Example:

php

```php
<?php
    $student = array("name" => "Amina", "age" => 20);
    echo $student["name"]; // Output: Amina
?>
```

Useful when data has labels (like name, age, price).

3. Multidimensional Arrays:

Arrays **inside other arrays**.

Used to represent **tables or complex data**.

Example:

php

```php
<?php
    $marks = array(
        "Ali" => array("Math" => 90, "Science" => 85),
        "Sara" => array("Math" => 95, "Science" => 80)
    );
    echo $marks["Ali"]["Science"]; // Output: 85
?>
```

Helpful for tabular data like student marks, products, etc.

Summary Table:

Type	Index Type	Example Index	Use Case
Indexed Array	Numeric (0, 1, 2…)	`$array[0]`	List of items
Associative Array	Named keys	`$array["name"]`	Key-value data
Multidimensional	Array of arrays	`$array[0][1]`	Table-like data

2.5 Creating Array, Accessing Array, Manipulating Arrays, Displaying arrays…..use of for..each as loop:

1. Creating Arrays in PHP:

Arrays can be created using the `array()` function or `[]` (short syntax).

Examples:

php

```php
$colors = array("Red", "Green", "Blue");
$fruits = ["Apple", "Banana", "Mango"];
```

2. Accessing Array Elements:

Elements are accessed using their **index** or **key**.

php

```php
echo $colors[1]; // Output: Green
```

php

```
$person = ["name" => "Ali", "age" => 25];
echo $person["name"]; // Output: Ali
```

3. Manipulating Arrays:

You can **add**, **modify**, or **remove** array elements.

Add:

php

```
$colors[] = "Yellow"; // Adds Yellow to the array
```

Modify:

php

```
$colors[0] = "Pink"; // Changes Red to Pink
```

Remove:

php

```
unset($colors[1]); // Removes Green
```

4. Displaying Arrays:

Use `print_r()` or `var_dump()` for quick display:

php

```
print_r($colors);
```

For custom display, use a loop.

5. Use of `foreach` Loop:

`foreach` is used to iterate over **arrays easily**.

Indexed Array:

php

```
foreach ($colors as $color) {
    echo $color . "<br>";
}
```

Associative Array:

php

```
foreach ($person as $key => $value) {
    echo "$key: $value <br>";
}
```

Summary:

Arrays store multiple values.
`foreach` is the best way to loop through arrays.
Arrays can be created, accessed, changed, and displayed easily in PHP.

2.6 Using Array Functions:

Using Array Functions in PHP:

PHP provides many **built-in array functions** to work with arrays easily — such as **adding, removing, sorting, counting**, etc.

Commonly Used Array Functions:

`count()`:

Returns the **number of elements** in an array.

php

```
$colors = ["Red", "Green", "Blue"];
echo count($colors); // Output: 3
```

`array_push()`:

Adds elements to the **end** of an array.

php

```
array_push($colors, "Yellow");
```

`array_pop()`:

Removes the **last** element from an array.

php

```
array_pop($colors);
```

`array_merge()`:

Combines two or more arrays into one.

php

```
$fruits = ["Apple", "Banana"];
$veggies = ["Carrot", "Tomato"];
$items = array_merge($fruits, $veggies);
```

`sort()`:

Sorts an array in **ascending order**.

php

```
sort($colors);
```

Why Use Array Functions?

Save time and effort
Clean and optimized code
Make array handling easier

Example:

php

```
$numbers = [4, 2, 8, 1];
sort($numbers);
foreach ($numbers as $num) {
    echo $num . " ";
}
```

// Output: 1 2 4 8

2.7 Including and Requiring Files- use of Include() and Require():

Including and Requiring Files in PHP:

PHP allows you to **reuse code** by including other PHP files into your current script using **`include()`** and **`require()`**.

1. Why Use `include()` or `require()`?

Avoid code repetition.
Keep your code organized (e.g., separating header, footer, config files).
Easy to manage large projects.

2. `include()` Function:

Includes and evaluates a file.
Continues execution even if the file is not found (shows a warning).

Example:

```
php
    include("header.php");
    echo "Main Page Content";
```

3. `require()` Function:

Includes and evaluates a file.
Stops execution if the file is not found (shows a fatal error).

Example:

```
php
    require("config.php");
```

Key Differences:

Function	If file not found	Use Case
`include()`	Shows warning, continues	Non-critical files (e.g., sidebar)
`require()`	Fatal error, stops	Critical files (e.g., DB config)

4. Best Practices:

Use `require()` for important files (like database connections).
Use `include()` for optional parts (like footer, sidebar).

2.8 Implicit and Explicit Casting in PHP:

Type Casting in PHP:

Type casting means converting a variable from one data type to another. PHP supports both:

Implicit Casting (automatic)
Explicit Casting (manual)

1. Implicit Casting (Type Juggling):

PHP **automatically converts** data types when needed.
Happens during operations like arithmetic or string concatenation.

Example:

php

```
$x = "5" + 10; // "5" is automatically cast to integer
echo $x; // Output: 15
```

Useful when combining strings and numbers.

2. Explicit Casting (Manual Conversion):

You **manually convert** a value using casting syntax.
Format: `(data_type) $variable`

Example:

php

```
$val = "123.45";
$intVal = (int)$val;
echo $intVal; // Output: 123
```

Ensures the variable is treated as a specific type.

3. Common Casting Types:

Type	Syntax
Integer	`(int)$var`
Float	`(float)$var`
String	`(string)$var`
Boolean	`(bool)$var`
Array	`(array)$var`

Why It Matters:

Prevents bugs when working with mixed types.
Helps control how data behaves in operations.

Unit III

3. Using Functions , Class- Objects, Forms in PHP

3.1 Functions in PHP: Defining and calling a function, Returning Value from function
3.2 Date and Time in PHP: Date and Time Functions
3.3 Strings in PHP: String Functions
3.4 Objects in PHP: What is Class & Object, Creating and accessing a Class & Object
3.5 Object properties, object methods, Overloading, inheritance, Constructor and Destructor
3.6 Creating HTML Form
3.7 Handling HTML Form data in PHP

3.1 Functions in PHP: Defining and calling a function, Returning Value from function:

Functions in PHP:

A **function** in PHP is a block of code that performs a specific task. It helps in **code reusability** and **modular programming**.

1. Defining a Function:

Use the `function` keyword to define a function.

Syntax:

```
php
    function functionName() {
        // code to execute
    }
```

Example:

```
php
    function greet() {
        echo "Hello, welcome!";
    }
```

2. Calling a Function:

Once defined, you can **call** the function by using its name followed by `()`.

Example:

```
php
    greet();  // Output: Hello, welcome!
```

3. Returning a Value from a Function:

A function can return a value using the `return` statement.

Example:

```
php
    function add($a, $b) {
        return $a + $b;
    }
    $result = add(5, 3);
```

```
echo $result;  // Output: 8
```

You can store the return value in a variable or use it directly.

Why Use Functions?

Reduces code repetition
Improves readability
Makes debugging easier

3.2 Date and Time in PHP: Date and Time Functions:

Date and Time in PHP:

PHP provides **built-in functions** to work with **date and time**, such as getting the current date, formatting it, or calculating time differences.

1. `date()` Function:

Used to **format** the current date and time.
Accepts format characters like `d` (day), `m` (month), `Y` (year), `H` (hour), `i` (minutes), etc.

Example:

php

```
echo date("d-m-Y");  // Output: 20-04-2025
echo date("h:i A");  // Output: 11:45 AM
```

2. `time()` Function:

Returns the **current Unix timestamp** (number of seconds since Jan 1, 1970).

Example:

php

```
echo time(); // Output: 1713604800 (example)
```

3. `mktime()` Function:

Creates a timestamp for a specific **date and time**.

php

```
$timestamp = mktime(0, 0, 0, 12, 25, 2025);
echo date("d-m-Y", $timestamp); // Output: 25-12-2025
```

4. `strtotime()` Function:

Converts a **human-readable date** into a timestamp.

php

```
$date = strtotime("next Friday");
echo date("d-m-Y", $date);
```

Why Use Date & Time Functions?

Display current time or date
Store timestamps
Schedule events or deadlines

Work with time zones

3.3 Strings in PHP: String Functions:

Strings in PHP:

A **string** is a sequence of characters used to store and manipulate text. PHP provides many **built-in string functions** to handle strings easily.

Common String Functions in PHP:

1. **`strlen()` –** Get String Length

   Returns the number of characters in a string.

   php

   ```
   $str = "Hello";
   echo strlen($str); // Output: 5
   ```

2. **`strtoupper()` and `strtolower()` –** Change Case

   Convert strings to upper or lower case.

   php

   ```
   echo strtoupper("hello"); // Output: HELLO
   echo strtolower("WORLD"); // Output: world
   ```

3. **`strpos()` –** Find Position of Substring

   Returns the position of the first occurrence of a substring.

   php

   ```
   echo strpos("Hello World", "World"); // Output: 6
   ```

4. **`str_replace()` –** Replace Text

   Replaces all occurrences of a word or character in a string.

   php

   ```
   echo str_replace("World", "PHP", "Hello World"); // Output: Hello PHP
   ```

5. **`substr()` –** Extract Part of a String

   Returns a part of a string.

   php

   ```
   echo substr("Hello PHP", 6); // Output: PHP
   ```

Why Use String Functions?

Useful for formatting and validating input
Helpful in search, replace, and text manipulation
Makes working with text easier and faster

3.4 Objects in PHP: What is Class & Object, Creating and accessing a Class & Object:

Objects in PHP:

PHP supports **Object-Oriented Programming (OOP)** using **classes** and **objects**. This approach helps organize code using real-world concepts.

1. What is a Class?

A **class** is a **template or blueprint** for creating objects.
It defines **properties (variables)** and **methods (functions)**.

```
php
    class Car {
        public $brand = "Toyota";

        public function start() {
            echo "Car is starting...";
        }
    }
```

2. What is an Object?

An **object** is an **instance of a class**.
It can **access properties and methods** of the class.

3. Creating a Class & Object:

Example:

```
    php
        // Define a class
        class Person {
            public $name;

            public function sayHello() {
                echo "Hello, my name is " . $this->name;
            }
        }

        // Create object
        $person1 = new Person();

        // Access property
        $person1->name = "Ayesha";

        // Call method
        $person1->sayHello(); // Output: Hello, my name is Ayesha
```

Key Terms:

Term	Description
`class`	Defines the structure (blueprint)
`object`	Instance of the class
`$this`	Refers to the current object
`->`	Used to access object members

Benefits of OOP in PHP:

Reusability of code
Better organization
Easier maintenance

3.5 Object properties, object methods, Overloading, inheritance, Constructor and Destructor:

1. Object Properties:

Variables defined inside a class.
Represent **characteristics** of an object.

php

```
class Car {
    public $brand = "Toyota";
}
```

2. Object Methods:

Functions defined inside a class.
Represent **actions** of an object.

php

```
class Car {
    public function start() {
        echo "Car started!";
    }
}
```

3. Constructor:

A special method that runs **automatically** when an object is created.
Declared using `__construct()`.

php

```
class Student {
    public $name;
    function __construct($n) {
        $this->name = $n;
    }
}
```

4. Destructor:

Runs **automatically** when an object is destroyed.
Declared using `__destruct()`.

php

```
function __destruct() {
```

```
        echo "Object destroyed.";
    }
```

5. Inheritance:

A class can **inherit** properties and methods from another class using the `extends` keyword.

```
php
    class Animal {
        public function sound() {
            echo "Animal sound";
        }
    }

    class Dog extends Animal {
        public function bark() {
            echo "Barking";
        }
    }
```

6. Overloading (Magic Methods):

Allows dynamic handling of properties or methods using **magic methods** like `__get()`, `__set()`, `__call()`.

```
php
    function __get($name) {
        echo "Trying to access $name";
    }
```

Summary Table:

Concept	Purpose
Properties	Store object data (variables)
Methods	Perform actions (functions)
Constructor	Auto-runs when object is created
Destructor	Auto-runs when object is destroyed
Inheritance	Reuse code from another class
Overloading	Handle undefined properties/methods

3.6 Creating HTML Form:

Creating HTML Form:

An **HTML form** is used to **collect user input** (like name, email, password) and send it to a server (often handled by PHP).

1. Basic Syntax of a Form:

```
html
```

```
<form action="submit.php" method="post">
    <!-- Form elements go here -->
</form>
```

`action`: The page where form data is sent.
`method`: How data is sent (`GET` or `POST`).

2. Common Form Elements:

html

```
<form action="submit.php" method="post">
    Name: <input type="text" name="username">><br>
    Email: <input type="email" name="email">><br>
    Password: <input type="password" name="password">><br>
    <input type="submit" value="Submit">
</form>
```

`<input>`: Used for text, email, password, etc.
`name`: Important for accessing data in PHP.

3. `GET` vs `POST` Methods:

Method	Description
`GET`	Sends data via URL, visible
`POST`	Sends data securely, not visible

Purpose of HTML Forms:

Login pages
Signup forms
Contact forms
Surveys, feedback, etc.

Output Example:

A form on a web page where users can **enter details and submit** them to a PHP file.

3.7 Handling HTML Form data in PHP:

Handling HTML Form Data in PHP:

When a user submits a form, PHP can **collect and process** that data using `$_GET` or `$_POST` superglobals depending on the form method.

1. Create HTML Form:

html

```
<form action="formhandler.php" method="post">
    Name: <input type="text" name="username">><br>
    <input type="submit" value="Submit">
</form>
```

`action="formhandler.php"` – data is sent to this PHP file.
`method="post"` – data is sent using POST method.

2. Handle Form Data in PHP:

In `formhandler.php`:

```
php
<?php
    $name = $_POST['username'];
    echo "Hello, " . $name;
?>
```

`$_POST['username']` – fetches the data entered in the form.
Use `$_GET[]` if method is `get`.

3. Using isset() to Avoid Errors:

```
php
if (isset($_POST['username'])) {
    $name = $_POST['username'];
    echo "Welcome, $name";
}
```

Prevents errors if the form is not submitted yet.

4. Security Tip – Use `htmlspecialchars()`:

```
php
$name = htmlspecialchars($_POST['username']);
```

Protects from XSS (Cross Site Scripting).

Summary:

Step	Description
Create HTML Form	With input fields & submit button
Use `$_POST` or `$_GET`	To access form data in PHP
Sanitize Input	For security using `htmlspecialchars()`

Unit IV

4. Database Handling Using PHP with MySQL

4.1 Introduction to MySQL: Database terms, Data Types
4.2 Accessing MySQL –Using MySQL Client and Using phpMyAdmin
4.3 MySQL Commands
4.4 Using PHP with MySQL: PHP MySQL Functions
4.5 Connecting to MySQL and Selecting the Database
4.6 Executing Simple Queries
4.7 Retrieving Query Results
4.8 Counting Returned Records
4.9 Updating Records with PHP

4.1 Introduction to MySQL: Database terms, Data Types:

Introduction to MySQL:

MySQL is a popular **open-source relational database management system (RDBMS)** used to store, manage, and retrieve data efficiently. It works with PHP to build dynamic websites and applications.

1. Common Database Terms:

Term	Description
Database	A collection of organized data.
Table	Stores data in rows and columns.
Row	A single record in a table.
Column	A field that holds specific data (e.g., name, age).
Primary Key	Uniquely identifies each row.
Foreign Key	Links one table to another.
Query	A command used to interact with the database.

2. Common MySQL Data Types:

Data Type	Description
INT	Integer numbers(e.g., age, ID)
VARCHAR	Variable-length string(e.g., name, email)
TEXT	Large text data
DATE	Stores date(YYYY-MM-DD)
DATETIME	Stores date and time
FLOAT/DOUBLE	Decimal numbers
BOOLEAN/TINYINT(1)	True/False values(1 or 0)

Example:

sql

```
CREATE TABLE students (
    id INT PRIMARY KEY,
    name VARCHAR(100),
```

```
        dob DATE,
        marks FLOAT
);
```

Summary:

MySQL stores data in structured form using tables.
Different data types are used based on the kind of data (numbers, text, date).
Understanding these basics is essential for building a strong database foundation.

4.2 Accessing MySQL –Using MySQL Client and Using phpMyAdmin:

Accessing MySQL:

You can access and manage MySQL databases in two main ways:

1. Using MySQL Client (Command Line Interface):

A **text-based interface** to interact with MySQL directly.
Useful for quick and powerful database management.

Steps:

1. Open Command Prompt or Terminal
2. Login to MySQL:
   bash
   mysql -u root –p
3. Enter your password
4. Now you can run SQL commands like:
   sql
   SHOW DATABASES;
   USE mydatabase;
   SELECT * FROM students;

Fast and flexible but requires knowledge of SQL.

2. Using phpMyAdmin (Web Interface):

A **browser-based GUI tool** for MySQL.
Easier for beginners and provides visual tools.

Steps:

1. Start your local server (XAMPP, WAMP, etc.)
2. Open browser and go to:
   http://localhost/phpmyadmin
3. Login with username/password (default: root / no password)
4. You can:
   Create/edit databases and tables
   Run SQL queries
   Import/export data

User-friendly and no SQL knowledge required for basic operations.

Summary Table:

Method	Interface	Best For
MySQL Client	Command-line	Developers, quick access
phpMyAdmin	Web-based GUI	Beginners, visual tools

4.3 MySQL Commands:

MySQL Commands:

MySQL commands are **SQL (Structured Query Language) statements** used to interact with a database to create, manage, and retrieve data from it.

1. Database Commands:

Create Database:

sql

```
CREATE DATABASE college;
```

Use Database:

sql

```
USE college;
```

Show Databases:

sql

```
SHOW DATABASES;
```

2. Table Commands:

Create Table:

sql

```
CREATE TABLE students (
    id INT PRIMARY KEY,
    name VARCHAR(100),
    marks FLOAT
);
```

Show Tables:

sql

```
SHOW TABLES;
```

3. Data Manipulation Commands:

Insert Data:

sql

```
INSERT INTO students VALUES (1, 'Ayaan', 85.5);
```

Select Data:

sql

```
SELECT * FROM students;
```

Update Data:

sql

```
UPDATE students SET marks = 90 WHERE id = 1;
```

Delete Data:

sql

```
DELETE FROM students WHERE id = 1;
```

Summary Table:

Command	Purpose
`CREATE`	To create database/table
`INSERT`	To add new data
`SELECT`	To retrieve data
`UPDATE`	To modify existing data
`DELETE`	To remove data

4.4 Using PHP with MySQL: PHP MySQL Functions:

Using PHP with MySQL:

PHP can connect to MySQL to **store**, **retrieve**, and **manage data** from a database. This is useful for creating dynamic websites like login forms, contact forms, etc.

1. Connecting to MySQL:

php

```
$conn = mysqli_connect("localhost", "root", "", "college");
```

`localhost`: Server name
`root`: Username
`""`: Password (empty by default in local setup)
`"college"`: Database name

2. PHP MySQL Functions:

Function	Purpose
`musqli_connect()`	Connect to MySQL database
`mysqli_query()`	Execute SQL queries (SELECT, INSERT, etc.)
`mysqli_fetch_assoc()`	Fetch query result as an associative array
`mysqli_close()`	Close the database connection
`mysqli_error()`	Display error if query fails

3. Example: Insert & Display Data:

php

```
$conn = mysqli_connect("localhost", "root", "", "college");
$sql = "INSERT INTO students (id, name, marks) VALUES (1, 'Ayaan', 85)";
```

```
mysqli_query($conn, $sql);

$result = mysqli_query($conn, "SELECT * FROM students");
while($row = mysqli_fetch_assoc($result)) {
    echo $row['name'] . " - " . $row['marks'] . "<br>";
}

mysqli_close($conn);
```

Summary:

PHP uses `mysqli_` functions to:

Connect to a database
Run queries
Fetch results
Close the connection

4.5 Connecting to MySQL and Selecting the Database:

Connecting to MySQL and Selecting the Database in PHP:

To work with MySQL in PHP, you first need to **connect to the MySQL server** and then **select a database** to run queries on.

1. Using `mysqli_connect()` Function:

php

```
$conn = mysqli_connect("localhost", "root", "", "college");
```

"localhost" – MySQL server (usually local)
"root" – MySQL username
"" – Password (empty for local servers like XAMPP/WAMP)
"college" – Name of the database

2. Checking Connection:

php

```
if (!$conn) {
    die("Connection failed: " . mysqli_connect_error());
}
echo "Connected successfully!";
```

This ensures the script stops if the connection fails and shows an error message.

3. Selecting a Database (Optional if included in connection):

You can also select the database separately:

php

```
mysqli_select_db($conn, "college");
```

Summary Table:

Function	Purpose
`mysqli_connect()`	Connects to MySQL server
`mysqli_select_db()`	Selects a database (if not passed in connect)
`mysqli_connect_error()`	Shows connection error if any

Example Output:

Connected successfully!

4.6 Executing Simple Queries:

Executing Simple Queries in PHP with MySQL:

To interact with a MySQL database from PHP, you use **SQL queries**. These queries can **insert, select, update, or delete** data from the database using PHP functions.

1. Using `mysqli_query()`:

This function is used to execute SQL queries in PHP:

php

```
mysqli_query($connection, $query);
```

2. Example: INSERT Query:

php

```
$conn = mysqli_connect("localhost", "root", "", "college");

$sql = "INSERT INTO students (id, name, marks) VALUES (1, 'Ayesha', 88)";
mysqli_query($conn, $sql);
```

This will insert a new record into the `students` table.

3. Example: SELECT Query:

php

```
$result = mysqli_query($conn, "SELECT * FROM students");

while($row = mysqli_fetch_assoc($result)) {
    echo $row['name'] . " - " . $row['marks'] . "<br>";
}
```

This retrieves all records and displays student name and marks.

4. Checking if Query was Successful:

php

```
if (mysqli_query($conn, $sql)) {
    echo "Query executed successfully";
} else {
    echo "Error: " . mysqli_error($conn);
}
```

Summary Table:

Function	Use
`mysqli_query()`	Executes any SQL command
`mysqli_fetch_assoc()`	Fetches result row as array (for SELECT)
`mysqli_error()`	Returns error message, if any

4.7 Retrieving Query Results:

Retrieving Query Results in PHP (MySQLi):

After running a `SELECT` query in PHP, you can retrieve and display data using **result-handling functions** like `mysqli_fetch_assoc()`.

1. Step-by-Step Process:

1. Run a SELECT Query:

php

```
$conn = mysqli_connect("localhost", "root", "", "college");

$sql = "SELECT * FROM students";
$result = mysqli_query($conn, $sql);
```

2. Check if Results Exist:

php

```
if (mysqli_num_rows($result) > 0) {
    // Data exists
}
```

3. Fetch and Display Results:

php

```
while($row = mysqli_fetch_assoc($result)) {
    echo "ID: " . $row['id'] . " - Name: " . $row['name'] . " - Marks: " .
  $row['marks'] . "<br>";
}
```

2. Common Result Functions:

Function	Description
`mysqli_query()`	Executes the SQL query
`mysqli_num_rows()`	Returns number of rows from SELECT result

`mysqli_fetch_assoc()`	Fetches a row as an associative array
`mysqli_fetch_array()`	Fetches row as both indexed and assoc. array

3. Example Output:

ID: 1 - Name: Ayesha - Marks: 88
ID: 2 - Name: Zaid - Marks: 92

Summary:

Use `SELECT` to retrieve data.
`mysqli_fetch_assoc()` is most commonly used for clean associative data.
Always check if data exists before fetching.

4.8 Counting Returned Records:

Counting Returned Records in PHP (MySQLi):

When you run a **`SELECT` query**, you can count how many records (rows) were returned using the **`mysqli_num_rows()`** function.

1. Step-by-Step Example:

php

```
$conn = mysqli_connect("localhost", "root", "", "college");

$sql = "SELECT * FROM students";
$result = mysqli_query($conn, $sql);

// Count records
$count = mysqli_num_rows($result);

echo "Total students: " . $count;
```

This displays the total number of rows fetched from the `students` table.

2. Function Used:

Function	Purpose
`mysqli_num_rows()`	Returns the number of rows in a result set

3. Use Case:

To show total results to the user
To check if any data was returned:

php

```
if (mysqli_num_rows($result) > 0) {
    echo "Data found!";
} else {
    echo "No data available.";
```

```
}
```

Summary:

`mysqli_num_rows()` is used **after a SELECT query**.
It helps in displaying totals or checking if any records were returned.

4.9 Updating Records with PHP:

Updating Records with PHP (MySQLi):

In PHP, you can **update existing records** in a MySQL database using the `UPDATE` SQL query and the `mysqli_query()` function.

1. Syntax of UPDATE Statement:

sql

```sql
UPDATE table_name SET column1 = value1 WHERE condition;
```

`SET` specifies the new value
`WHERE` filters which row(s) to update

2. PHP Example: Update a Record:

php

```php
$conn = mysqli_connect("localhost", "root", "", "college");

$sql = "UPDATE students SET marks = 95 WHERE id = 1";

if (mysqli_query($conn, $sql)) {
    echo "Record updated successfully";
} else {
    echo "Error updating record: " . mysqli_error($conn);
}
```

This updates the marks of the student with `id = 1`.

3. Points to Remember:

Always use a **WHERE clause** to avoid updating all rows.
Use **form input** for dynamic updates:

php

```php
$id = $_POST['id'];
$newMarks = $_POST['marks'];
$sql = "UPDATE students SET marks = $newMarks WHERE id = $id";
```

Summary Table:

Function	Use
`mysqli_query()`	Executes the update SQL query
`mysqli_error()`	Shows error if update fails